Einblick in RTAL SYNTECH

> **Aktualisierung V1.0.2:** Die frühere Aussage, die finale Quellcodebasis sei noch nicht geprüft, ist überholt. Ein statisches Audit des hochgeladenen V1.0.0-FINAL-ZIP wurde durchgeführt. Historische Benchmarkwerte bleiben ohne Laufzeitlogs unbestätigt.

Hohe Klangqualität durch eine effiziente DSP-Architektur

Technische Architektur- und Performance-Analyse — RTAL SYNTECH V1.0.0 FINAL
Dokumentrevision: 1.0.2 (mit Quellcode-Audit)
Datum: 11. November 2026
Plattform: ESP32-S3 / JC4827W543 (MAIN)
Abtastrate: 32.000 Hz | **Blockgröße:** 256 Audioframes | **Release-Konfiguration:** 6-stimmige Polyphonie

> **Geltungsbereich und Beleglage:** Dieses Dokument verbindet die von Joakim Eriksson öffentlich dokumentierte Scintilla-Architektur mit Beobachtungen aus der RTAL-SYNTECH-Entwicklung und den Hardwaretests. Historische Messwerte werden ausdrücklich gekennzeichnet. Die Release-Konfiguration (32 kHz / 256 Frames / sechs Noten) folgt der Dokumentation und den Entwicklungsaufzeichnungen zu V1.0.0 FINAL. Das finale Firmware-ZIP wurde mittlerweile statisch geprüft; ein unabhängig reproduzierter Hardwarebenchmark liegt nicht vor. Messungen auf unterschiedlichen Chips, mit unterschiedlichen Sounds, aktiven Stimmen und Effekteinstellungen sind **nicht unmittelbar vergleichbar**.

Zusammenfassung

RTAL SYNTECH überträgt eine vom Elka Synthex inspirierte Stimmenarchitektur auf ein kompaktes Touch-Instrument mit ESP32-S3. Die zentrale Erkenntnis aus Joakim Erikssons **Scintilla** lautet: Hohe Klangqualität und Effizienz auf eingebetteten Systemen schließen sich nicht aus. Entscheidend ist ein Audio-Rechenpfad, der unnötige Arbeit vermeidet — durch Integer-/Festkommaverarbeitung, Lookup-Tabellen, gezielte Parametervorbereitung, spezialisierte Modulationspfade sowie bewusste Speicher- und CPU-Nutzung.

Die wichtigsten Erkenntnisse betreffen **die Begrenzung des algorithmischen Rechenaufwands** und **die Speicherlokalität**. In einer früheren RTAL-Diskussion wurde eine deutliche Beschleunigung durch die Verlagerung eines häufig verwendeten Reverb-Puffers aus dem PSRAM in internes SRAM beschrieben. Die Umsetzung der SRAM-Priorisierung in V1.0.0 ist inzwischen im Quellcode **bestätigt**; das zugehörige historische A/B-Messprotokoll liegt jedoch nicht vor. Die früher genannten Reverb-Zeiten erscheinen deshalb nachfolgend ausschließlich als *unbestätigte Entwicklungsnotizen* und nicht als nachgewiesene Release-Performance.

1. Ursprung und Urheberschaft

Joakim Eriksson entwickelte Scintilla, eine in Integer-C implementierte Synthesizer-Engine im Stil des Elka Synthex. Das öffentliche Scintilla-Repository beschreibt eine Stimmen-Engine mit **zwei PolyBLEP-Oszillatoren, Rauschgenerator, fünf auf Zero-Delay-Feedback basierenden Filtermodi, zwei Hüllkurven, zwei LFOs, Glide, Oszillatorsynchronisation, Ringmodulation und Cross-Modulation**. Die darüberliegende Synthesizer-Ebene verwaltet Layer, Stimmenmodi, Chorus, Reverb und Limiter. Außerdem erläutert das Projekt Klangvergleiche mit der Referenzimplementierung `synthex-web`.

Wichtige Originalquellen:

- Joakim Eriksson: https://github.com/joakimeriksson
- Scintilla-Projekt: https://github.com/joakimeriksson/scintilla-fm1

- Frühere ESP32-/ESP32-P4-Experimente: https://github.com/joakimeriksson/esp32

Würdigung: Scintilla und Joakims frühere Synthex-Arbeiten bilden die wesentliche technische Grundlage und Inspiration für RTAL SYNTECH. RTAL-spezifische GUI, MIDI-Integration, Presetspeicherung, Board-Portierung, Echtzeit-Taskplanung, Anpassungen und Performance-Arbeiten müssen als eigenständige Integrationsleistungen von der ursprünglichen DSP-Entwicklung unterschieden werden.

Lizenzhinweis: Das öffentliche Repository `scintilla-fm1` ist mit GPL-3.0 gekennzeichnet. Es beschreibt außerdem die Herkunft von Code aus `synthex-web` und weiteren Komponenten. Bei einer Veröffentlichung abgeleiteten Codes müssen Lizenzen und Urhebervermerke **anhand genau der Upstream-Version geprüft werden, die tatsächlich in RTAL SYNTECH übernommen wurde**. Dabei sind die einschlägigen Hinweise, Lizenztexte und Quellcodebereitstellungspflichten zu beachten. Dieses Dokument erklärt die Technik, **ersetzt jedoch keine Lizenzprüfung**.

2. Was macht die Engine so effizient?

2.1 Integer- und Festkommaarithmetik

Die Engine setzt auf Integer-C-DSP statt auf eine universell mit Fließkommazahlen arbeitende Synthesizer-Implementierung. Festkommaarithmetik kann die Operationen pro Sample besser planbar machen und wiederholte Fließkommakonvertierungen oder aufwendige mathematische Funktionen vermeiden. Der tatsächliche Vorteil hängt jedoch von Prozessor, Compiler, Zahlenformat und Übersteuerungsbehandlung ab: Festkomma ist **nicht bei jeder Operation automatisch schneller**.

Entwurfsprinzip: Die zentrale Sample-Schleife sollte klein, übersichtlich und hinsichtlich ihrer Laufzeit begrenzt bleiben. Wertebereiche müssen so gewählt werden, dass Überläufe, Clipping und Koeffizientengenauigkeit beherrschbar sind.

2.2 Lookup-Tabellen statt wiederholter aufwendiger Berechnungen

Die Upstream-Datei `engine/sc_tables.h` enthält in dieser Upstream-Version Tabellen für Tonhöhe, Cutoff, Sinus sowie logarithmische und exponentielle Zusammenhänge, die für **44,1 kHz** erzeugt wurden. Das ist **nicht** die Betriebsabtastrate von SYNTECH V1.0.0 (32 kHz). Vorberechnete Werte benötigen Speicher, reduzieren dafür jedoch wiederholte mathematische Operationen in der Echtzeit-Audioberechnung.

Bedeutung für SYNTECH: Aufbau und Interpolation der Tabellen, Cache-Lokalität und Speicherplatzierung können wichtiger sein als der nominelle CPU-Takt. Für die Portierung auf 32 kHz müssen abtastratenabhängige Tabellen, Oszillator-Phasenschritte, Filterkoeffizienten und Zeitkonstanten auf 32 kHz abgestimmt werden. Der finale Quellcode bestätigt die zentralen 32-kHz-Anpassungen; eine vollständige numerische Verifikation aller Tabellen und Koeffizienten steht aus.

2.3 Keine unnötige Berechnung inaktiver Stimmen

Bei einem polyphonen Synthesizer hängt die tatsächliche Rechenlast von der Zahl der momentan bearbeiteten Stimmen ab — nicht nur von der beworbenen maximalen Polyphonie. Werden inaktive Stimmen übersprungen, verbrauchen ihre Oszillatoren, Hüllkurven und Filter nach dem vollständigen Ausklingen keine weiteren Rechenzyklen. In V1.0.0 werden vollständig inaktive Stimmen in `sc_render()` nachweislich übersprungen.

Entwurfsprinzip: Stimmenzuweisung, Release-Phasen, Effektsends und Note-Off-Verhalten gehören zusammen. Eine Stimme darf nicht einfach ausgelassen werden, solange ihre Ausklingphase noch hörbar ist.

2.4 Parametervorbereitung statt ständiger Neuberechnung

Scintilla trennt parameterabhängige Vorbereitungen von der eigentlichen Syntheseberechnung pro Sample. Bei unseren RTAL-Untersuchungen identifizierten wir `sc_reprep()` als wichtigen Vorbereitungspfad und diskutierten die Bündelung redundanter Neuberechnungen bei vielen gleichzeitig eintreffenden Touch- oder MIDI-Ereignissen.

Die **Häufigkeit von Koeffizientenaktualisierungen** bleibt ein technischer Kompromiss. Es wäre falsch zu behaupten, sämtliche Parameter würden grundsätzlich nur einmal pro Audioblock neu berechnet. Schnelle Filterhüllkurven, Oszillatorwechselwirkungen und Modulationen mit Audiorate können häufigere Aktualisierungen erfordern. Die aktuelle öffentliche Scintilla-Dokumentation weist ausdrücklich darauf hin, dass schnelle Filterhüllkurven-Attacks in anspruchsvollen Passagen nahezu bei jedem Sample eine Koeffizientenberechnung auslösen können.

2.5 Spezialisierte Berechnungspfade

Die Upstream-Engine unterscheidet normale Berechnungspfade von besonders aufwendigen Fällen wie Cross-Modulation pro Sample, PWM, Oszillatorsynchronisation und schnellen Filteränderungen. Dadurch kann ein einfacher gehaltener Akkord wesentlich weniger CPU-Zeit benötigen als ein Preset mit intensiver Modulation. **Die reine Stimmenzahl ist deshalb keine ausreichende Performance-Kennzahl.**

2.6 Klare Aufgabenverteilung im Echtzeitbetrieb

Bei RTAL SYNTECH wird die Audioberechnung soweit praktikabel von aufwendigen UI-, Touch-, MIDI- und SD-Karten-Operationen abgeschirmt. Eine FreeRTOS-Befehlswarteschlange bildet eine Schnittstelle zwischen den Ereigniserzeugern und der Audio-Engine. Das unterstützt eine kontrollierte Taskplanung, beseitigt aber keine CPU-Engpässe: Eine große Zahl von Parameteränderungen kann weiterhin einzelne Audioblöcke überlasten und XRUNs verursachen.

Echtzeitregel: Unvorhersehbare Dateisystemzugriffe, Speicherallokationen, Display-Rendering und Debug-Ausgaben gehören nicht in die zeitkritische Audio-Callback-Verarbeitung.

3. Warum der Speicherort entscheidend ist: das Reverb-Beispiel

Während der Entwicklung wurde ein Reverb-Delay-Array mit folgendem Speicherbedarf diskutiert:

```
int32_t buf[2048][8];   // 2048 × 8 × 4 = 65.536 Bytes = 64 KiB
```

Diese Rechnung belegt zunächst nur die **Größe dieses Beispielarrays**. Das finale ZIP bestätigt ein solches Reverb-Array (64 KiB). Es wird bevorzugt intern im SRAM angelegt; bei Allokationsfehlern erfolgt ein PSRAM-Fallback. Häufig angesprochene Delay-Puffer reagieren stärker auf PSRAM-Zugriffszeiten und Cache-Fehlzugriffe als selten verwendete Presettabellen.

Frühere Diskussion — **nicht unabhängig verifiziert**	Genannter Wert
Reverb-Verarbeitung vor der Verlagerung	ca. 500–600 µs
Reverb-Verarbeitung nach der Verlagerung	ca. 154–182 µs
Aus den Bereichsgrenzen abgeleitete relative Reduktion	ca. 64–74 % (je nach Wertepaar)

Beleggrenze: Diese Zahlen wurden in der Diskussion vom 4. November 2026 wiedergegeben. Die konkreten SRAM-Allokationspfade der V1.0.0 sind geprüft; die vollständigen historischen A/B-Messprotokolle und Versuchsbedingungen liegen jedoch nicht vor. Sie dürfen deshalb **nicht als**

bestätigter A/B-Benchmark der Release-Firmware veröffentlicht werden. Ebenso ist eine eindeutige Kausalzuordnung allein zum Speicherort nicht belegt.

Praktische Erkenntnis: Häufig verwendete Puffer, ihren realen Speicherbereich in Map-Datei/Heap-Diagnose und ihre Laufzeit unter identischen Preset- und Build-Bedingungen prüfen. Internes SRAM ist begrenzt und wird ebenfalls für DMA-fähige Puffer, Stacks und andere zeitkritische Daten benötigt.

4. Echtzeit-Zeitbudget — finale Konfiguration

Das Audioblock-Zeitfenster ergibt sich unmittelbar aus den tatsächlichen Release-Einstellungen:

```
Abtastrate = 32.000 Samples/s
Blockgröße = 256 Frames
T_Block = 256 / 32.000 s = 0,008 s = 8,000 ms = 8.000 µs
```

8.000 µs sind das zeitliche Blockintervall, nicht automatisch 8.000 µs frei verfügbares DSP-Budget. I²S-/DMA-Bedienung, Taskplanung, Interrupts und sonstige Arbeiten erfordern eine Sicherheitsreserve. Welche Anteile in einer gemessenen Renderzeit enthalten sind, hängt von den Messgrenzen in der Firmware ab.

4.1 Historische Werte aus *früheren* Firmwareständen

Für **V0.4B** wurde ein Hardwarelog mit **44,1 kHz und 128 Frames** besprochen (damals rund **2.902 µs** Blockintervall). Dabei wurden etwa **174–176 µs** bei geringer Last, bis zu **421 µs** bei aktiver Berechnung und ein genannter **Spitzenwert von 665 µs** protokolliert; in diesem Lauf gab es keine XRUNs und Queue-Verluste. Diese Werte gehören ausschließlich zu dieser früheren Version und dürfen **nicht als Messwerte der finalen 32-kHz-/256-Frame-Firmware** ausgegeben werden.

Die in der Erstfassung enthaltenen Stimmenskalierungszahlen (332 / 816 / 1.464 / 2.836 µs bei 1 / 4 / 8 / 16 Stimmen) sind **entfernt**, weil kein verlässlich zuordenbares Messprotokoll mit identischer Konfiguration und definierter Zählweise vorlag. Daraus lässt sich keine Polyphonieaussage ableiten.

4.2 Upstream-Benchmarks sind keine ESP32-S3-Benchmarks

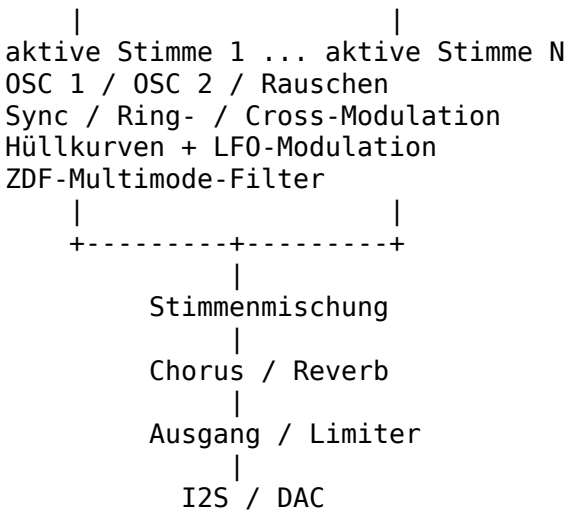
Die öffentliche README von `scintilla-fm1` enthält Schätzungen zum Instruktionsaufwand für **256-Frame-Blöcke bei 44,1 kHz auf der FM-1-Plattform** mit deren Hardware-/Emulatorannahmen und ausgewählten Presets. Sie zeigen erhebliche Unterschiede je nach Modulation und Hüllkurven-Attack. Die Werte lassen sich nicht als Prozentsätze oder Renderzeiten auf RTAL SYNTECH mit ESP32-S3 übertragen.

Eine zuvor diskutierte **ESP32-P4-Angabe von etwa 80 µs** hat hier keine reproduzierbar dokumentierte Preset-, Stimmen-, Effekt- und Messdefinition. Sie bleibt deshalb **kein verifizierter Benchmark**. ESP32-P4 und ESP32-S3 besitzen zudem deutlich unterschiedliche Prozessor- und Speichersysteme.

5. Synthesizer-Architektur und Kostentreiber

Vereinfachtes Verarbeitungsdiagramm:

```
MIDI DIN / Touch-GUI / Preset-Befehle
                  |
          Ereignis-/Befehlsqueue
                  |
      Synthesizerzustand + Stimmenzuweisung
                  |
        +---------+---------+
```



Das Diagramm ist vereinfacht. Die tatsächliche Reihenfolge und Taskzuordnung sind im separaten Quellcode-Audit beschrieben.

Besonders rechenintensive Presets kombinieren häufig mehrere Faktoren: viele gleichzeitig aktive Stimmen oder Layer, Cross-Modulation und Synchronisation, kontinuierlich veränderbare Oszillatorwellenformen, schnelle Filterhüllkurven, schnelle LFO-Modulation und aktive Effekte. Daher muss ein Belastungstest **sowohl die Klangkomplexität als auch die Stimmenzahl** erfassen.

6. RTAL-spezifische Entwicklungsleistungen rund um die Engine

RTAL SYNTECH integriert die Synthese-Engine in ein ESP32-S3-Instrument mit Touch-Bedienung und übernimmt nicht einfach Joakims ursprüngliche Benutzeroberfläche. Zum Projektkontext der V1.0.0 gehören:

Bereich	RTAL SYNTECH V1.0.0
Hauptcontroller	ESP32-S3, JC4827W543 Touch-Board
Audio	32 kHz, 256 Audioframes pro Block, 32-Bit-I2S-Pfad
Polyphonie	6 Noten auf MAIN
Touch-Display	NV3041A QSPI, 480 × 272, LVGL 8.4 / GT911
DIN-MIDI	RX GPIO17, TX GPIO18, 31.250 Baud
MIDI-Steuerung	MOPHO-Modus mit CC32-Bankwahl, STANDARD-Modus mit direkter CC-Zuordnung, MIDI Learn
Preset-Verwaltung	128 SD-Presetplätze: 000–079 Factory, 080–127 User; `.rtal`
Echtzeit-Integration	Eigenständige Audioverarbeitung und über eine Queue übermittelte UI-/MIDI-Befehle

Dies sind **Beschreibungen der Projekt- und Release-Konfiguration** und keine Ergebnisse einer für dieses Dokument unabhängig durchgeführten Code- oder Hardwareprüfung.

7. Checkliste für zukünftige DSP-Performance-Tests

Bei künftigen Änderungen sollten auf derselben Hardware und mit demselben Firmware-Build folgende Angaben erfasst werden:

1. Build-Kennung, CPU-Frequenz, Compiler-Flags und Speicherbelegung.

2. Abtastrate, Blockgröße sowie genaue Stimmen- und Layer-Konfiguration.
3. Preset, Oszillatorwechselwirkungen, Filtermodus, Effekte und Modulationstiefe.
4. Durchschnittliche, maximale und hohe Perzentilwerte der Block-Rechenzeit über eine ausreichend lange Messung.
5. XRUNs, Queue-Überläufe, Clipping und verlorene MIDI-Ereignisse.
6. Schnelle Parameteränderungen getrennt von Tests mit stabil gehaltenen Akkorden.
7. Kaltstart, SD-Preset-Ladevorgänge und gleichzeitige GUI-Bedienung.
8. A/B-Vergleiche mit **jeweils nur einer veränderten Einflussgröße**.

Ein einfacher Kennwert für die CPU-Zeitreserve lautet:

```
Zeitreserve ≈ 1 - (gemessene maximale Renderzeit / 8.000 µs)
```

Ein negatives Ergebnis bedeutet, dass die Blockzeit überschritten wurde. Ein positives Ergebnis ist nur eine Näherung für die verbleibende Reserve, da dabei nicht gemessene Zusatzkosten unberücksichtigt bleiben.

8. Welche Schlussfolgerungen sind gerechtfertigt?

Zur Architektur belegt: Erikssons Scintilla ist bewusst als kompakter Integer-C-Synthesizer mit Lookup-Tabellen, PolyBLEP-Oszillatoren und performancebewussten Berechnungspfaden entwickelt. Die öffentliche Dokumentation beschreibt deutliche Unterschiede im Rechenaufwand verschiedener Presets.

Historische RTAL-Entwicklungsdiskussion (für das Final-Release nicht verifiziert): Nach einer SRAM-/PSRAM-Änderung wurde eine starke Beschleunigung des Reverbs beschrieben. Ohne originale, vergleichbare Messprotokolle und finalen Quellcode ist das ein plausibler Optimierungsansatz, **aber kein bestätigter quantitativer Messbefund**.

Release-Konfiguration: RTAL SYNTECH V1.0.0 auf dem MAIN-Board ist ein sechsstimmiges ESP32-S3-Instrument mit 32 kHz und 256 Audioframes pro Block. Nicht verifizierte Angaben zu mehr Stimmen oder zur Rechenzeit eines ESP32-P4 belegen keine höhere Polyphonie der Release-Firmware.

Durch dieses Dokument nicht nachgewiesen: eine reproduzierbare Worst-Case-Latenzmessung der finalen Firmware; acht beziehungsweise sechzehn Stimmen auf dem veröffentlichten ESP32-S3; eine lückenlose Kausalanalyse aller Compiler- und DSP-Optimierungen; oder die vollständige Einhaltung aller Lizenzanforderungen Datei für Datei.

9. Danksagung und weiterführende Quellen

Ursprüngliche Syntheseentwicklung und Inspiration: Joakim Eriksson — Scintilla und verwandte Synthex-Projekte.

RTAL-Integration und Weiterentwicklung: RealTimeAudioLab — Anpassung an die ESP32-S3-Plattform, Touch-Oberfläche, MIDI-Modi, SD-Preset-System, Echtzeit-Taskplanung sowie die hier dokumentierten Performance-Untersuchungen.

Quellen:

1. Joakim Eriksson, *Scintilla for M-VAVE FM-1*, https://github.com/joakimeriksson/scintilla-fm1 — Projekt-README, Engine-Beschreibung, numerische Vergleiche, Instruktionsaufwand und GPL-3.0-Lizenz.
2. Joakim Eriksson, *ESP32 experiments*, https://github.com/joakimeriksson/esp32 — ergänzender historischer Plattformkontext.
3. RTAL-SYNTECH-Entwicklungsgespräche vom **30. Oktober und 4. November 2026** — historische Hardwaremessungen und eine *nicht unabhängig überprüfte* Diskussion zur Speicherplatzierung, mit den oben genannten Einschränkungen. Finale Release-Konfiguration gemäß der Projektdokumentation vom 11. November 2026.

Dokumentationsrevision 1.0.1 (an V1.0.0 FINAL angepasste Korrekturfassung) — 11. November 2026. Diese Datei ist als technische Ergänzung für GitHub vorgesehen. Zahlenangaben werden ausdrücklich auf die jeweilige Beleglage und Testkonfiguration begrenzt.

Ergänzung V1.0.2: Prüfung des finalen Firmware-Quellcodes

Das Quellcode-ZIP `RTAL_SYNTECH_V1_0_0_FINAL(2).ZIP` wurde am 11. November 2026 **statisch analysiert**. Diese Prüfung bestätigt:

- **32.000 Hz / 256 Stereo-Frames / 8.000 µs:** `RTAL_AUDIO_SAMPLE_RATE`, `FRAMES`, `SC_FS` und I²S sind konsistent eingestellt; `static_assert` prüft DSP gegen I²S.
- **32-kHz-DSP:** `sc_tables.h` kennzeichnet neu generierte Pitch-/Cutoff-Tabellen. LFO-Takt, Hüllkurven-Zeiten, 96-Sample-Filterkoeffizientenblöcke, Chorus, Reverb und Limiter wurden für 32 kHz angepasst.
- **DSP-Ablauf:** Core-1-AudioTask (Priorität 23), FreeRTOS-Befehlsqueue, gebündelte Parametervorbereitung, Stimmenrendering, optionale Chorus-/Reverb-Stufen, Limiter, Q15-zu-I²S32-Konvertierung.
- **Reverb tatsächlich optimiert:** Der **64-KiB-Reverbring** wird **zuerst im internen SRAM angefordert**, mit PSRAM als Fallback. Die Aussage, das Reverb liege in FINAL grundsätzlich im PSRAM, ist falsch. Die tatsächlich erreichte Platzierung zeigt das Boot-Log `[FIX5 MEMORY]`.
- **Speicher:** Engine-Zustand wird ebenfalls zunächst intern, ersatzweise im PSRAM alloziert. Chorus-Delay (8.192 B), Limiter-Delay (1.536 B Nutzdaten), Audio- und Mixpuffer sowie LVGL-DMA-Puffer (57.600 B) sind im Code identifizierbar. Eine vollständige Heap-/Flash-Speicherbilanz verlangt eine Linker-Map und Messungen.
- **Polyphonie korrekt beschreiben:** Die Engine hält **8 Voice-Slots je Layer** vor, nicht exakt sechs insgesamt. Die für SYNTECH festgelegte sechsnotige Betriebsweise ist von internen Slots und den zusätzlichen Layer-Stimmen zu unterscheiden.

Prüfgrenzen: Der Code wurde nicht für die Zielhardware gebaut oder auf Hardware gemessen; LUT-Werte wurden nicht vollständig numerisch validiert. Die historischen Reverb-A/B-Zeiten sind daher weiterhin keine durch dieses ZIP bestätigten Laufzeitmessungen. Einzelheiten: `RTAL_SYNTECH_V1_0_0_FINAL_Quellcode_Audit_DSP_Speicher_32k_DE.md`.